

The 'sopra-models' package

Documentation for the 'sopra-models' package | Version v1.0.00

2. December 2019

Florian Sihler (florian.sihler@uni-ulm.de)

1 General

1.1 Why, wherefore, whence?

This $\text{\LaTeX}$ 2_ε-package was written as part of the Sopra project in the winter semester of 2019 and summer semester of 2020, and serves as the basis for presenting (UML) diagrams and models for *Team 20*. This documentation was created together with `sopra-base.cls` and the `sopra-documentation.sty` package.

Important: A large part of this package is based on `tikz-uml` and is documented here: <https://perso.ensta-paris.fr/~kielbasi/tikzuml/>.

The `sopra-listings` package is used to visualize the individual code snippets. It is not relevant for using the package.

The corresponding package should also be embedded in this document: .

1.2 Dependencies

This package includes the following packages:

- `tikz`
- `float`
- `calc`
- `newfloat`
- `ifthen`
- `caption`
- `xstring`
- `pgfopts`

All of these packages should be part of any common $\text{\LaTeX}$ -distribution. Furthermore, TikZ uses the following libraries: `backgrounds`, `arrows`, `shapes`, `fit`, `shadows` and `decorations.markings`.

1.3 Installation

The package is not distributed as a `.dtx` file, which is why the following options are available:

- The package can be placed in the same directory as the document. In this case, the inclusion command reads:
`\usepackage{sopra-models}`
- The package can be placed in a subdirectory, or in a directory shipped alongside the document. In this case, it is specified via the (relative) path:
`\usepackage{./My/Path/to/sopra-models}`
- The package can be placed (via a symlink or similar) into your own `texmf` tree. For example, on Linux using `texlive`, the package can be placed here: `~/texmf/tex/latex/`. The directory can be created and then updated using `texhash ~/texmf`. The package can now be used like any other installed package:
`\usepackage{sopra-models}`

1.4 Further Notes

In version v1.0.00 ($\backslash\text{thesomversion} \rightarrow \text{P. 2}$) there are no further notes.

2 Package Configuration

2.1 Accepted Parameters

This package itself does not accept any further options!

3 Commands and Environments

Note that the prefix env@ only indicates the nature of an environment and is not part of the actual identifier!

3.1 General Commands

- ▷ $\backslash\text{thesomversion}$
- Returns the current version of the package. This gives: $\backslash\text{thesomversion}$: v1.0.00
Note: via $\backslash\text{value}\{\text{somversion}\}$ the version can be obtained as a 4-digit number: 1000.

3.2 Floats

- ▷ $\text{env@model}[\textit{Placement}]$
- A float, completely analogous to env@figure and env@table .

Example:

```
 $\backslash\text{begin}\{\text{model}\}$   
   $\backslash\text{centering}$   
  I'm a model.  
   $\backslash\text{caption}[\text{And I'm actually shorter.}]\{\text{I am the title.}\}$   
 $\backslash\text{end}\{\text{model}\}$ 
```

Gives:

I'm a model.

Modell 3.1: I am the title.

These can be listed via $\backslash\text{listofmodel}$:

Alle Modelle

3.1 And I'm actually shorter.	2
3.2 A complete model.	3
3.3 Another complete model.	3

By embedding the modified TikZ-UML, a whole range of models becomes possible. For example, the one in Modell 3.2 (which is scaled to the page width). Another example can be found in Modell 3.3.

The diagram illustrates the architecture of a database management system and its dependencies. It features several components and artifacts:

- Hardware access layer** (component) and **oracle** (artifact): The hardware access layer depends on the oracle artifact, indicated by a dashed arrow labeled "Dependency that expresses the compatibility of the two interfaces." The hardware access layer provides a "File access" provided interface (half-circle) which the oracle artifact requires (dashed line with a square).
- Subscriber management** (component): This component has a "Complex port" (dashed line with a square) and an "Unsorted access" provided interface (half-circle). It also has a "Sorted access" required interface (dashed line with a circle) and a "Storage medium" provided interface (half-circle).
- Database management system** (component): This is a large component that contains two subsystems:
 - Input/output system** (subsystem): It has a "Simple port" (dashed line with a square) and a "Composition connector" (dashed line with a circle).
 - XML interface** (component): It has a "Component" (dashed line with a square) and a "Composition connector" (dashed line with a circle).The database management system provides a "Hardware access" provided interface (half-circle) which the hardware access layer requires (dashed line with a square).
- List generator** (component): This component has a "Sorted access" provided interface (half-circle) which the subscriber management component requires (dashed line with a circle). It also has a "Dependency" (dashed line with a square) and an "Implementation relationship" (dashed line with a square).
- Pagerator** and **Typesetting program**: These are external components that depend on the list generator component, indicated by dashed arrows.
- Lister v2 ear** (artifact): This artifact is a manifestation of the list generator component, indicated by a dashed arrow labeled "Artifact".

Legend:

- Component: `<<component>>`
- Artifact: `<<artifact>>`
- Subsystem: `<<subsystem>>`
- Complex port: Dashed line with a square
- Simple port: Dashed line with a circle
- Composition connector: Dashed line with a circle
- Component: Dashed line with a square

```

    usecaseDiagram
        actor User
        actor Server
        usecase Hexxagon as Hexxagon
        usecase Hexxagon as ShowMainMenu[Show main menu]
        usecase Hexxagon as CreateLobby[Create lobby]
        usecase Hexxagon as ManageLobbies[Manage lobbies]
        usecase Hexxagon as JoinLeaveLobby[Join/leave lobby]
        usecase Hexxagon as PlayHexxagon[Play Hexxagon]
        usecase Hexxagon as DisplayGame[Display game]
        usecase Hexxagon as MoveStone[Move stone]
        usecase Hexxagon as ShowGameOver[Show Game Over screen]

        User -- ShowMainMenu
        User -- CreateLobby
        User -- ManageLobbies
        User -- JoinLeaveLobby
        User -- PlayHexxagon
        User -- DisplayGame
        User -- MoveStone
        User -- ShowGameOver

        Server -- JoinLeaveLobby
        Server -- MoveStone

        ShowMainMenu <-->|<<include>>| ManageLobbies
        CreateLobby <-->|<<include>>| ManageLobbies
        ManageLobbies <-->|<<include>>| JoinLeaveLobby
        ManageLobbies <-->|<<include>>| PlayHexxagon
        PlayHexxagon <-->|<<include>>| DisplayGame
        PlayHexxagon <-->|<<include>>| MoveStone
        PlayHexxagon <-->|<<include>>| ShowGameOver
    
```

The diagram illustrates the use cases for the Hexxagon game, organized within a system boundary. The actors involved are the User and the Server. The use cases are represented by ovals and are interconnected by dashed arrows indicating dependencies, with some arrows labeled with the stereotype <<include>>.

Actors:

- User
- Server

Use Cases:

- Show main menu
- Create lobby
- Manage lobbies
- Join/leave lobby
- Play Hexxagon
- Display game
- Move stone
- Show Game Over screen

Relationships:

- The User is associated with all use cases: Show main menu, Create lobby, Manage lobbies, Join/leave lobby, Play Hexxagon, Display game, Move stone, and Show Game Over screen.
- The Server is associated with Join/leave lobby and Move stone.
- Dependencies (indicated by dashed arrows):
 - Show main menu includes Manage lobbies (<<include>>).
 - Create lobby includes Manage lobbies (<<include>>).
 - Manage lobbies includes Join/leave lobby (<<include>>).
 - Manage lobbies includes Play Hexxagon (<<include>>).
 - Play Hexxagon includes Display game (<<include>>).
 - Play Hexxagon includes Move stone (<<include>>).
 - Play Hexxagon includes Show Game Over screen (<<include>>).

Modell 3.3: Another complete model.